第 15 章

原子变量与非阻塞同步机制

在 java.util.concurrent 包的许多类中，例如 Semaphore 和 ConcurrentLinkedQueue，都提供了比 synchronized 机制更高的性能和可伸缩性。本章将介绍这种性能提升的主要来源：原子变量和非阻塞的同步机制。

近年来，在并发算法领域的大多数研究都侧重于非阻塞算法，这种算法用底层的原子机器指令（例如比较并交换指令）代替锁来确保数据在并发访问中的一致性。非阻塞算法被广泛地用于在操作系统和 JVM 中实现线程 / 进程调度机制、垃圾回收机制以及锁和其他并发数据结构。

与基于锁的方案相比，非阻塞算法在设计和实现上都要复杂得多，但它们在可伸缩性和活跃性上却拥有巨大的优势。由于非阻塞算法可以使多个线程在竞争相同的数据时不会发生阻塞，因此它能在粒度更细的层次上进行协调，并且极大地减少调度开销。而且，在非阻塞算法中不存在死锁和其他活跃性问题。在基于锁的算法中，如果一个线程在休眠或自旋的同时持有一个锁，那么其他线程都无法执行下去，而非阻塞算法不会受到单个线程失败的影响。从 Java 5.0 开始，可以使用原子变量类（例如 AtomicInteger 和 AtomicReference）来构建高效的非阻塞算法。

即使原子变量没有用于非阻塞算法的开发，它们也可以用做一种“更好的 volatile 类型变量”。原子变量提供了与 volatile 类型变量相同的内存语义，此外还支持原子的更新操作，从而使它们更加适用于实现计数器、序列发生器和统计数据收集等，同时还能比基于锁的方法提供更高的可伸缩性。

15.1 锁的劣势

通过使用一致的锁定协议来协调对共享状态的访问，可以确保无论哪个线程持有守护变量的锁，都能采用独占方式来访问这些变量，并且对变量的任何修改对随后获得这个锁的其他线程都是可见的。

现代的许多 JVM 都对非竞争锁获取和锁释放等操作进行了极大的优化，但如果有多个线程同时请求锁，那么 JVM 就需要借助操作系统的功能。如果出现了这种情况，那么一些线程将被挂起并且在稍后恢复运行[⊖]。当线程恢复执行时，必须等待其他线程执行完它们的时间片

⊖ 当线程在锁上发生竞争时，智能的 JVM 不一定会挂起线程，而是根据之前获取操作中对锁的持有时间长短来判断是使此线程挂起还是自旋等待。

以后，才能被调度执行。在挂起和恢复线程等过程中存在着很大的开销，并且通常存在着较长时间的中断。如果在基于锁的类中包含有细粒度的操作（例如同步容器类，在其大多数方法中只包含了少量操作），那么当在锁上存在着激烈的竞争时，调度开销与工作开销的比值会非常高。

与锁相比，volatile 变量是一种更轻量级的同步机制，因为在使用这些变量时不会发生上下文切换或线程调度等操作。然而，volatile 变量同样存在一些局限：虽然它们提供了相似的可见性保证，但不能用于构建原子的复合操作。因此，当一个变量依赖其他的变量时，或者当变量的新值依赖于旧值时，就不能使用 volatile 变量。这些都限制了 volatile 变量的使用，因此它们不能用来实现一些常见的工具，例如计数器或互斥体 (mutex)。[⊖]

例如，虽然自增操作 (++i) 看起来像一个原子操作，但事实上它包含了 3 个独立的操作——获取变量的当前值，将这个值加 1，然后再写入新值。为了确保更新操作不被丢失，整个的读－改－写操作必须是原子的。到目前为止，我们实现这种原子操作的唯一方式就是使用锁定方法，如第 2 章的 Counter 所示。

Counter 是线程安全的，并且在没有竞争的情况下能运行得很好。但在竞争的情况下，其性能会由于上下文切换的开销和调度延迟而降低。如果锁的持有时间非常短，那么当在不恰当的时间请求锁时，使线程休眠将付出很高的代价。

锁定还存在其他一些缺点。当一个线程正在等待锁时，它不能做任何其他事情。如果一个线程在持有锁的情况下被延迟执行（例如发生了缺页错误、调度延迟，或者其他类似情况），那么所有需要这个锁的线程都无法执行下去。如果被阻塞线程的优先级较高，而持有锁的线程优先级较低，那么这将是一个严重的问题——也被称为优先级反转 (Priority Inversion)。即使高优先级的线程可以抢先执行，但仍然需要等待锁被释放，从而导致它的优先级会降至低优先级线程的级别。如果持有锁的线程被永久地阻塞（例如由于出现了无限循环，死锁，活锁或者其他的活跃性故障），所有等待这个锁的线程就永远无法执行下去。

即使忽略这些风险，锁定方式对于细粒度的操作（例如递增计数器）来说仍然是一种高开销的机制。在管理线程之间的竞争时应该有一种粒度更细的技术，类似于 volatile 变量的机制，同时还要支持原子的更新操作。幸运的是，在现代的处理器中提供了这种机制。

15.2 硬件对并发的支持

独占锁是一项悲观技术——它假设最坏的情况（如果你不锁门，那么捣蛋鬼就会闯入并搞得一团糟），并且只有在确保其他线程不会造成干扰（通过获取正确的锁）的情况下才能执行下去。

对于细粒度的操作，还有另外一种更高效的方法，也是一种乐观的方法，通过这种方法可以在不发生干扰的情况下完成更新操作。这种方法需要借助冲突检查机制来判断在更新过程中是否存在来自其他线程的干扰，如果存在，这个操作将失败，并且可以重试（也可以不重试）。这种乐观的方法就好像一句谚语："原谅比准许更容易得到"，其中"更容易"在这里相当于

⊖ 虽然理论上可以基于 volatile 的语义来构造互斥体和其他同步器，但在实际情况中很难实现。

“更高效”。

在针对多处理器操作而设计的处理器中提供了一些特殊指令，用于管理对共享数据的并发访问。在早期的处理器中支持原子的测试并设置 (Test-and-Set)，获取并递增 (Fetch-and-Increment) 以及交换 (Swap) 等指令，这些指令足以实现各种互斥体，而这些互斥体又可以实现一些更复杂的并发对象。现在，几乎所有的现代处理器中都包含了某种形式的原子读 - 改 - 写指令，例如比较并交换 (Compare-and-Swap) 或者关联加载 / 条件存储 (Load-Linked/Store-Conditional)。操作系统和 JVM 使用这些指令来实现锁和并发的数据结构，但在 Java 5.0 之前，在 Java 类中还不能直接使用这些指令。

15.2.1 比较并交换

在大多数处理器架构（包括 IA32 和 Sparc）中采用的方法是实现一个比较并交换（CAS）指令。（在其他处理器中，例如 PowerPC，采用一对指令来实现相同的功能：关联加载与条件存储。）CAS 包含了 3 个操作数——需要读写的内存位置 V、进行比较的值 A 和拟写入的新值 B。当且仅当 V 的值等于 A 时，CAS 才会通过原子方式用新值 B 来更新 V 的值，否则不会执行任何操作。无论位置 V 的值是否等于 A，都将返回 V 原有的值。（这种变化形式被称为比较并设置，无论操作是否成功都会返回。）CAS 的含义是：“我认为 V 的值应该为 A，如果是，那么将 V 的值更新为 B，否则不修改并告诉 V 的值实际为多少”。CAS 是一项乐观的技术，它希望能成功地执行更新操作，并且如果有另一个线程在最近一次检查后更新了该变量，那么 CAS 能检测到这个错误。程序清单 15-1 中的 SimulatedCAS 说明了 CAS 语义（而不是实现或性能）。

程序清单 15-1 模拟 CAS 操作

```
@ThreadSafe
public class SimulatedCAS {
    @GuardedBy("this") private int value;

    public synchronized int get() { return value; }

    public synchronized int compareAndSwap(int expectedValue,
                                           int newValue) {
        int oldValue = value;
        if (oldValue == expectedValue)
            value = newValue;
        return oldValue;
    }

    public synchronized boolean compareAndSet(int expectedValue,
                                              int newValue) {
        return (expectedValue
                == compareAndSwap(expectedValue, newValue));
    }
}
```

当多个线程尝试使用 CAS 同时更新同一个变量时，只有其中一个线程能更新变量的值，而其他线程都将失败。然而，失败的线程并不会被挂起（这与获取锁的情况不同：当获取锁失

败时，线程将被挂起），而是被告知在这次竞争中失败，并可以再次尝试。由于一个线程在竞争 CAS 时失败不会阻塞，因此它可以决定是否重新尝试，或者执行一些恢复操作，也或者不执行任何操作。[⊖]这种灵活性就大大减少了与锁相关的活跃性风险（尽管在一些不常见的情况下仍然存在活锁风险——请参见 10.3.3 节）。

CAS 的典型使用模式是：首先从 V 中读取值 A，并根据 A 计算新值 B，然后再通过 CAS 以原子方式将 V 中的值由 A 变成 B（只要在这期间没有任何线程将 V 的值修改为其他值）。由于 CAS 能检测到来自其他线程的干扰，因此即使不使用锁也能够实现原子的读－改－写操作序列。

15.2.2 非阻塞的计数器

程序清单 15-2 中的 CasCounter 使用 CAS 实现了一个线程安全的计数器。递增操作采用了标准形式——读取旧的值，根据它计算出新值（加 1），并使用 CAS 来设置这个新值。如果 CAS 失败，那么该操作将立即重试。通常，反复地重试是一种合理的策略，但在一些竞争很激烈的情况下，更好的方式是在重试之前首先等待一段时间或者回退，从而避免造成活锁问题。

程序清单 15-2 基于 CAS 实现的非阻塞计数器

```
@ThreadSafe
public class CasCounter {
    private SimulatedCAS value;

    public int getValue() {
        return value.get();
    }

    public int increment() {
        int v;
        do {
            v = value.get();
        }
        while (v != value.compareAndSwap(v, v + 1));
        return v + 1;
    }
}
```

CasCounter 不会阻塞，但如果其他线程同时更新计数器，那么会多次执行重试操作[⊖]。（在实际情况中，如果仅需要一个计数器或序列生成器，那么可以直接使用 AtomicInteger 或 AtomicLong，它们能提供原子的递增方法以及其他算术方法。）

⊖ 如果在 CAS 失败时不执行任何操作，那么是一种明智的做法。在非阻塞算法中，例如 15.4.2 节中的链表队列算法，当 CAS 失败时，意味着其他线程已经完成了你想要执行的操作。

⊖ 理论上，如果其他线程在每次竞争 CAS 时总是获胜，那么这个线程每次都会重试，但在实际中，很少发生这种类型的饥饿问题。

初看起来，基于 CAS 的计数器似乎比基于锁的计数器在性能上更差一些，因为它需要执行更多的操作和更复杂的控制流，并且还依赖看似复杂的 CAS 操作。但实际上，当竞争程度不高时，基于 CAS 的计数器在性能上远远超过了基于锁的计数器，而在没有竞争时甚至更高。如果要快速获取无竞争的锁，那么至少需要一次 CAS 操作再加上与其他锁相关的操作，因此基于锁的计数器即使在最好的情况下也会比基于 CAS 的计数器在一般情况下能执行更多的操作。由于 CAS 在大多数情况下都能成功执行（假设竞争程度不高），因此硬件能够正确地预测 while 循环中的分支，从而把复杂控制逻辑的开销降至最低。

虽然 Java 语言的锁定语法比较简洁，但 JVM 和操作在管理锁时需要完成的工作却并不简单。在实现锁定时需要遍历 JVM 中一条非常复杂的代码路径，并可能导致操作系统级的锁定、线程挂起以及上下文切换等操作。在最好的情况下，在锁定时至少需要一次 CAS，因此虽然在使用锁时没有用到 CAS，但实际上也无法节约任何执行开销。另一方面，在程序内部执行 CAS 时不需要执行 JVM 代码、系统调用或线程调度操作。在应用级上看起来越长的代码路径，如果加上 JVM 和操作系统中的代码调用，那么事实上却变得更短。CAS 的主要缺点是，它将使调用者处理竞争问题（通过重试、回退、放弃），而在锁中能自动处理竞争问题（线程在获得锁之前将一直阻塞）。[⊖]

CAS 的性能会随着处理器数量的不同而变化很大。在单 CPU 系统中，CAS 通常只需要很少的时钟周期，因为不需要处理器之间的同步。在编写本书时，非竞争的 CAS 在多 CPU 系统中需要 10 到 150 个时钟周期的开销。CAS 的执行性能不仅在不同的体系架构之间变化很大，甚至在相同处理器的不同版本之间也会发生改变。生产厂商迫于竞争的压力，在接下来的几年内还会继续提高 CAS 的性能。一个很管用的经验法则是：在大多数处理器上，在无竞争的锁获取和释放的“快速代码路径”上的开销，大约是 CAS 开销的两倍。

15.2.3 JVM 对 CAS 的支持

那么，Java 代码如何确保处理器执行 CAS 操作？在 Java 5.0 之前，如果不编写明确的代码，那么就无法执行 CAS。在 Java 5.0 中引入了底层的支持，在 int、long 和对象的引用等类型上都公开了 CAS 操作，并且 JVM 把它们编译为底层硬件提供的最有效方法。在支持 CAS 的平台上，运行时把它们编译为相应的（多条）机器指令。在最坏的情况下，如果不支持 CAS 指令，那么 JVM 将使用自旋锁。在原子变量类（例如 java.util.concurrent.atomic 中的 AtomicXxx）中使用了这些底层的 JVM 支持为数字类型和引用类型提供一种高效的 CAS 操作，而在 java.util.concurrent 中的大多数类在实现时则直接或间接地使用了这些原子变量类。

15.3 原子变量类

原子变量比锁的粒度更细，量级更轻，并且对于在多处理器系统上实现高性能的并发代码来说是非常关键的。原子变量将发生竞争的范围缩小到单个变量上，这是你获得的粒度最细的

⊖ 事实上，CAS 最大的缺陷在于难以围绕着 CAS 正确地构建外部算法。

情况（假设算法能够基于这种细粒度来实现）。更新原子变量的快速（非竞争）路径不会比获取锁的快速路径慢，并且通常会更快，而它的慢速路径肯定比锁的慢速路径快，因为它不需要挂起或重新调度线程。在使用基于原子变量而非锁的算法中，线程在执行时更不易出现延迟，并且如果遇到竞争，也更容易恢复过来。

原子变量类相当于一种泛化的 volatile 变量，能够支持原子的和有条件的读－改－写操作。AtomicInteger 表示一个 int 类型的值，并提供了 get 和 set 方法，这些 Volatile 类型的 int 变量在读取和写入上有着相同的内存语义。它还提供了一个原子的 compareAndSet 方法（如果该方法成功执行，那么将实现与读取 / 写入一个 volatile 变量相同的内存效果），以及原子的添加、递增和递减等方法。AtomicInteger 表面上非常像一个扩展的 Counter 类，但在发生竞争的情况下能提供更高的可伸缩性，因为它直接利用了硬件对并发的支持。

共有 12 个原子变量类，可分为 4 组：标量类（Scalar）、更新器类、数组类以及复合变量类。最常用的原子变量就是标量类：AtomicInteger、AtomicLong、AtomicBoolean 以及 AtomicReference。所有这些类都支持 CAS，此外，AtomicInteger 和 AtomicLong 还支持算术运算。（要想模拟其他基本类型的原子变量，可以将 short 或 byte 等类型与 int 类型进行转换，以及使用 floatToIntBits 或 doubleToLongBits 来转换浮点数。）

原子数组类（只支持 Integer、Long 和 Reference 版本）中的元素可以实现原子更新。原子数组类为数组的元素提供了 volatile 类型的访问语义，这是普通数组所不具备的特性——volatile 类型的数组仅在数组引用上具有 volatile 语义，而在其元素上则没有。（15.4.3 节和 15.4.4 节将讨论其他类型的原子变量。）

尽管原子的标量类扩展了 Number 类，但并没有扩展一些基本类型的包装类，例如 Integer 或 Long。事实上，它们也不能进行扩展：基本类型的包装类是不可修改的，而原子变量类是可修改的。在原子变量类中同样没有重新定义 hashCode 或 equals 方法，每个实例都是不同的。与其他可变对象相同，它们也不宜用做基于散列的容器中的键值。

15.3.1 原子变量是一种“更好的 volatile”

在 3.4.2 节中，我们使用了一个指向不可变对象的 volatile 引用来原子地更新多个状态变量。这个示例依赖于“先检查再运行”，但这种特殊的情况下，竞争是无害的，因为我们并不关心是否会偶尔地丢失更新操作。而在大多数情况下，这种“先检查再运行”不会是无害的，并且可能破坏数据的一致性。例如，第 4 章中的 NumberRange 既不能使用指向不可变对象的 volatile 引用来安全地实现上界和下界，也不能使用原子的整数来保存这两个边界。由于有一个不变性条件限制了两个数值，并且它们无法在同时更新时还维持该不变性条件，因此如果在数值范围类中使用 volatile 引用或者多个原子整数，那么将出现不安全的“先检查再运行”操作序列。

可以将 OneValueCache 中的技术与原子引用结合起来，并且通过对指向不可变对象（其中保存了下界和上界）的引用进行原子更新以避免竞态条件。在程序清单 15-3 的 CasNumber-

Range 中使用了 AtomicReference 和 IntPair 来保存状态，并通过使用 compare-AndSet，使它在更新上界或下界时能避免 NumberRange 的竞态条件。

程序清单 15-3 通过 CAS 来维持包含多个变量的不变性条件

```
public class CasNumberRange {
    @Immutable
    private static class IntPair {
final int lower;  // 不变性条件：lower <= upper
        final int upper;
        ...
    }
    private final AtomicReference<IntPair> values =
        new AtomicReference<IntPair>(new IntPair(0, 0));

    public int getLower() { return values.get().lower; }
    public int getUpper() { return values.get().upper; }

    public void setLower(int i) {
        while (true) {
            IntPair oldv = values.get();
            if (i > oldv.upper)
                throw new IllegalArgumentException(
                   "Can't set lower to " + i + " > upper");
            IntPair newv = new IntPair(i, oldv.upper);
            if (values.compareAndSet(oldv, newv))
                return;
        }
    }
    // 对 setUpper 采用类似的方法
}
```

15.3.2 性能比较：锁与原子变量

为了说明锁和原子变量之间的可伸缩性差异，我们构造了一个测试基准，其中将比较伪随机数字生成器 (PRNG) 的几种不同实现。在 PRNG 中，在生成下一个随机数字时需要用到上一个数字，所以在 PRNG 中必须记录前一个数值并将其作为状态的一部分。

程序清单 15-4 和程序清单 15-5 给出了线程安全的 PRNG 的两种实现，一种使用 ReentrantLock, 另一种使用 AtomicInteger。测试程序将反复调用它们，在每次迭代中将生成一个随机数字（在此过程中将读取并修改共享的 seed 状态），并执行一些仅在线程本地数据上执行的“繁忙”迭代。这种方式模拟了一些典型操作，以及一些在共享状态以及线程本地状态上的操作。

程序清单 15-4 基于 ReentrantLock 实现的随机数生成器

```
@ThreadSafe
public class ReentrantLockPseudoRandom extends PseudoRandom {
    private final Lock lock = new ReentrantLock(false);
```

```
    private int seed;

    ReentrantLockPseudoRandom(int seed) {
        this.seed = seed;
    }

    public int nextInt(int n) {
        lock.lock();
        try {
            int s = seed;
            seed = calculateNext(s);
            int remainder = s % n;
            return remainder > 0 ? remainder : remainder + n;
        } finally {
            lock.unlock();
        }
    }
}
```

程序清单 15-5 基于 AtomicInteger 实现的随机数生成器

```
@ThreadSafe
public class AtomicPseudoRandom extends PseudoRandom {
    private AtomicInteger seed;

    AtomicPseudoRandom(int seed) {
        this.seed = new AtomicInteger(seed);
    }

    public int nextInt(int n) {
        while (true) {
            int s = seed.get();
            int nextSeed = calculateNext(s);
            if (seed.compareAndSet(s, nextSeed)) {
                int remainder = s % n;
                return remainder > 0 ? remainder : remainder + n;
            }
        }
    }
}
```

图 15-1 和图 15-2 给出了在每次迭代中工作量较低以及适中情况下的吞吐量。如果线程本地的计算量较少，那么在锁和原子变量上的竞争将非常激烈。如果线程本地的计算量较多，那么在锁和原子变量上的竞争会降低，因为在线程中访问锁和原子变量的频率将降低。

从这些图中可以看出，在高度竞争的情况下，锁的性能将超过原子变量的性能，但在更真实的竞争情况下，原子变量的性能将超过锁的性能[⊖]。这是因为锁在发生竞争时会挂起线程，从而降低了 CPU 的使用率和共享内存总线上的同步通信量。（这类似于在生产者－消费者设计

⊖ 这个结论在其他领域同样成立：当交通拥堵时，交通信号灯能够实现更高的吞吐量，而在低拥堵时，环岛能实现更高的吞吐量。在以太网中使用的竞争机制在低通信流量的情况下运行得很好，但在高通信流量的情况下，令牌环网络中的令牌传递机制则表现得更好。

中的可阻塞生产者，它能降低消费者上的工作负载，使消费者的处理速度赶上生产者的处理速度。）另一方面，如果使用原子变量，那么发出调用的类负责对竞争进行管理。与大多数基于 CAS 的算法一样，AtomicPseudoRandom 在遇到竞争时将立即重试，这通常是一种正确的方法，但在激烈竞争环境下却导致了更多的竞争。

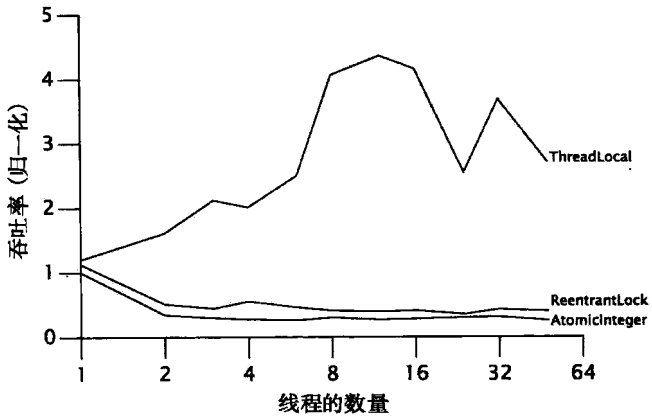

图 15-1 在竞争程度较高情况下的 Lock 与 AtomicInteger 的性能

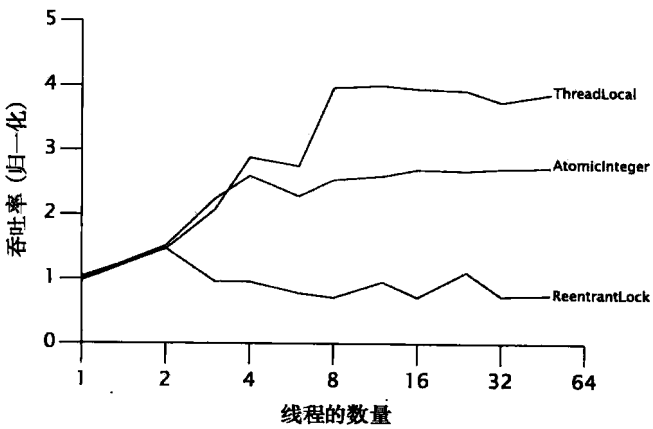

图 15-2 在竞争程度适中情况下的 Lock 与 AtomicInteger 的性能

在批评 AtomicPseudoRandom 写得太糟糕或者原子变量比锁更糟糕之前，应该意识到图 15-1 中竞争级别过高而有些不切实际：任何一个真实的程序都不会除了竞争锁或原子变量，其他什么工作都不做。在实际情况中，原子变量在可伸缩性上要高于锁，因为在应对常见的竞争程度时，原子变量的效率会更高。

锁与原子变量在不同竞争程度上的性能差异很好地说明了各自的优势和劣势。在中低程度的竞争下，原子变量能提供更高的可伸缩性，而在高强度的竞争下，锁能够更有效地避免竞

争。（在单 CPU 的系统上，基于 CAS 的算法在性能上同样会超过基于锁的算法，因为 CAS 在单 CPU 的系统上通常能执行成功，只有在偶然情况下，线程才会在执行读 - 改 - 写的操作过程中被其他线程抢占执行。）

在图 15-1 和图 15-2 中都包含了第三条曲线，它是一个使用 ThreadLocal 来保存 PRNG 状态的 PseudoRandom。这种实现方法改变了类的行为，即每个线程都只能看到自己私有的伪随机数字序列，而不是所有线程共享同一个随机数序列，这说明了，如果能够避免使用共享状态，那么开销将会更小。我们可以通过提高处理竞争的效率来提高可伸缩性，但只有完全消除竞争，才能实现真正的可伸缩性。

15.4 非阻塞算法

在基于锁的算法中可能会发生各种活跃性故障。如果线程在持有锁时由于阻塞 I/O，内存页缺失或其他延迟而导致推迟执行，那么很可能所有线程都不能继续执行下去。如果在某种算法中，一个线程的失败或挂起不会导致其他线程也失败或挂起，那么这种算法就被称为非阻塞算法。如果在算法的每个步骤中都存在某个线程能够执行下去，那么这种算法也被称为无锁 (Lock-Free) 算法。如果在算法中仅将 CAS 用于协调线程之间的操作，并且能正确地实现，那么它既是一种无阻塞算法，又是一种无锁算法。无竞争的 CAS 通常都能执行成功，并且如果有多个线程竞争同一个 CAS，那么总会有一个线程在竞争中胜出并执行下去。在非阻塞算法中通常不会出现死锁和优先级反转问题（但可能会出现饥饿和活锁问题，因为在算法中会反复地重试）。到目前为止，我们已经看到了一个非阻塞算法：CasCounter。在许多常见的数据结构中都可以使用非阻塞算法，包括栈、队列、优先队列以及散列表等，而要设计一些新的这种数据结构，最好还是由专家们来完成。

15.4.1 非阻塞的栈

在实现相同功能的前提下，非阻塞算法通常比基于锁的算法更为复杂。创建非阻塞算法的关键在于，找出如何将原子修改的范围缩小到单个变量上，同时还要维护数据的一致性。在链式容器类（例如队列）中，有时候无须将状态转换操作表示为对节点链接的修改，也无须使用 AtomicReference 来表示每个必须采用原子操作来更新的链接。

栈是最简单的链式数据结构：每个元素仅指向一个元素，并且每个元素也只被一个元素引用。在程序清单 15-6 的 ConcurrentStack 中给出了如何通过原子引用来构建栈的示例。栈是由 Node 元素构成的一个链表，其中栈顶作为根节点，并且在每个元素中都包含了一个值以及指向下一个元素的链接。push 方法创建一个新的节点，该节点的 next 域指向当前的栈顶，然后使用 CAS 把这个新节点放入栈顶。如果在开始插入节点时，位于栈顶的节点没有发生变化，那么 CAS 就会成功，如果栈顶节点发生了变化（例如由于其他线程在本线程开始之前插入或移除了元素），那么 CAS 将会失败，而 push 方法会根据栈的当前状态来更新节点，并且再次尝试。无论哪种情况，在 CAS 执行完成后，后栈仍会处于一致的状态。

程序清单 15-6 使用 Treiber 算法 (Treiber, 1986) 构造的非阻塞栈

```
@ThreadSafe
public class ConcurrentStack <E> {
    AtomicReference<Node<E>> top = new AtomicReference<Node<E>>();

    public void push(E item) {
        Node<E> newHead = new Node<E>(item);
        Node<E> oldHead;
        do {
            oldHead = top.get();
            newHead.next = oldHead;
        } while (!top.compareAndSet(oldHead, newHead));
    }

    public E pop() {
        Node<E> oldHead;
        Node<E> newHead;
        do {
            oldHead = top.get();
            if (oldHead == null)
                return null;
            newHead = oldHead.next;
        } while (!top.compareAndSet(oldHead, newHead));
        return oldHead.item;
    }

    private static class Node <E> {
        public final E item;
        public Node<E> next;

        public Node(E item) {
            this.item = item;
        }
    }
}
```

在 CasCounter 和 ConcurrentStack 中说明了非阻塞算法的所有特性：某项工作的完成具有不确定性，必须重新执行。在 ConcurrentStack 中，当构造表示新元素的 Node 时，我们希望当把这个新节点压入到栈时，其 next 引用的值仍然是正确的，同时也准备好在发生竞争的情况下重新尝试。

在像 ConcurrentStack 这样的非阻塞算法中都能确保线程安全性，因为 compareAndSet 像锁定机制一样，既能提供原子性，又能提供可见性。当一个线程需要改变栈的状态时，将调用 compareAndSet，这个方法与写入 volaitle 类型的变量有着相同的内存效果。当线程检查栈的状态时，将在同一个 AtomicReference 上调用 get 方法，这个方法与读取 volaitle 类型的变量有着相同的内存效果。因此，一个线程执行的任何修改结构都可以安全地发布给其他正在查看状态的线程。并且，这个栈是通过 compareAndSet 来修改的，因此将采用原子操作来更新 top 的引用，或者在发现存在其他线程干扰的情况下，修改操作将失败。

15.4.2 非阻塞的链表

到目前为止，我们已经看到了两个非阻塞算法，计数器和栈，它们很好地说明了 CAS 的基本使用模式：在更新某个值时存在不确定性，以及在更新失败时重新尝试。构建非阻塞算法的技巧在于：将执行原子修改的范围缩小到单个变量上。这在计数器中很容易实现，在栈中也很简单，但对于一些更复杂的数据结构，例如队列、散列表或树，则要复杂得多。

链接队列比栈更为复杂，因为它必须支持对头节点和尾结点的快速访问。因此，它需要单独维护的头指针和尾指针。有两个指针指向位于尾部的节点：当前最后一个元素的 next 指针，以及尾节点。当成功地插入一个新元素时，这两个指针都需要采用原子操作来更新。初看起来，这个操作无法通过原子变量来实现。在更新这两个指针时需要不同的 CAS 操作，并且如果第一个 CAS 成功，但第二个 CAS 失败，那么队列将处于不一致的状态。而且，即使这两个 CAS 都成功了，那么在执行这两个 CAS 之间，仍可能有另一个线程会访问这个队列。因此，在为链接队列构建非阻塞算法时，需要考虑到这两种情况。

我们需要使用一些技巧。第一个技巧是，即使在一个包含多个步骤的更新操作中，也要确保数据结构总是处于一致的状态。这样，当线程 B 到达时，如果发现线程 A 正在执行更新，那么线程 B 就可以知道有一个操作已部分完成，并且不能立即开始执行自己的更新操作。然后，B 可以等待（通过反复检查队列的状态）并直到 A 完成更新，从而使两个线程不会相互干扰。

虽然这种方法能够使不同的线程“轮流”访问数据结构，并且不会造成破坏，但如果一个线程在更新操作中失败了，那么其他的线程都无法再访问队列。要使得该算法成为一个非阻塞算法，必须确保当一个线程失败时不会妨碍其他线程继续执行下去。因此，第二个技巧是，如果当 B 到达时发现 A 正在修改数据结构，那么在数据结构中应该有足够多的信息，使得 B 能完成 A 的更新操作。如果 B“帮助”A 完成了更新操作，那么 B 可以执行自己的操作，而不用等待 A 的操作完成。当 A 恢复后再试图完成其操作时，会发现 B 已经替它完成了。

在程序清单 15-7 的 LinkedQueue 中给出了 Michael-Scott 提出的非阻塞链接队列算法中的插入部分 (Michael and Scott，1996)，在 ConcurrentLinkedQueue 中使用的正是该算法。在许多队列算法中，空队列通常都包含一个“哨兵 (Sentinel) 节点”或者“哑（Dummy）节点”，并且头节点和尾节点在初始化时都指向该哨兵节点。尾节点通常要么指向哨兵节点（如果队列为空），即队列的最后一个元素，要么（当有操作正在执行更新时）指向倒数第二个元素。图 15-3 给出了一个处于正常状态（或者说稳定状态）的包含两个元素的队列。

程序清单 15-7 Michael-Scott(Michael and Scott, 1996) 非阻塞算法中的插入算法

```
@ThreadSafe
public class LinkedQueue <E> {
    private static class Node <E> {
        final E item;
        final AtomicReference<Node<E>> next;

        public Node(E item, Node<E> next) {
            this.item = item;
            this.next = new AtomicReference<Node<E>>(next);
        }
```

```
    }

    private final Node<E> dummy = new Node<E>(null, null);
    private final AtomicReference<Node<E>> head
            = new AtomicReference<Node<E>>(dummy);
    private final AtomicReference<Node<E>> tail
            = new AtomicReference<Node<E>>(dummy);

    public boolean put(E item) {
        Node<E> newNode = new Node<E>(item, null);
        while (true) {
            Node<E> curTail = tail.get();
            Node<E> tailNext = curTail.next.get();
            if (curTail == tail.get()) {
                if (tailNext != null) {                                  A
                    // 队列处于中间状态，推进尾节点
                    tail.compareAndSet(curTail, tailNext);               B
                } else {
                    // 处于稳定状态，尝试插入新节点
                    if (curTail.next.compareAndSet(null, newNode)) {     C
                        // 插入操作成功，尝试推进尾节点
                        tail.compareAndSet(curTail, newNode);            D
                        return true;
                    }
                }
            }
        }
    }
}
```

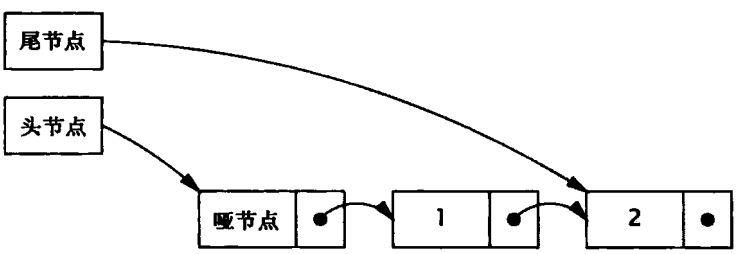

图 15-3 处于稳定状态并包含两个元素的对立

当插入一个新的元素时，需要更新两个指针。首先更新当前最后一个元素的 next 指针，将新节点链接到列表队尾，然后更新尾节点，将其指向这个新元素。在这两个操作之间，队列处于一种中间状态，如图 15-4 所示。在第二次更新完成后，队列将再次处于稳定状态，如图 15-5 所示。

实现这两个技巧时的关键点在于：当队列处于稳定状态时，尾节点的 next 域将为空，如果队列处于中间状态，那么 tail.next 将为非空。因此，任何线程都能够通过检查 tail.next 来获取队列当前的状态。而且，当队列处于中间状态时，可以通过将尾节点向前移动一个节点，从而结束其他线程正在执行的插入元素操作，并使得队列恢复为稳定状态。[⊖]

⊖ 在（Michael and Scott，1996）以及 (Herlihy and Shavit，2006) 中都给出了对算法正确性的完整分析。

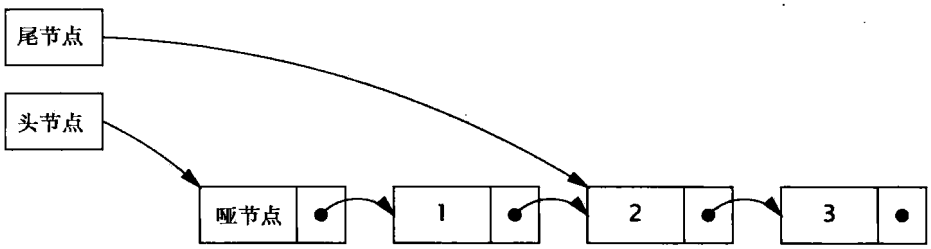

图 15-4　在插入过程中处于中间状态的对立

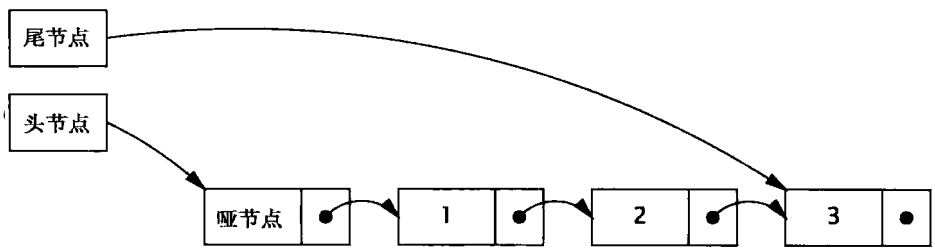

图 15-5　在插入操作完成后，队列再次处于稳定状态

LinkedQueue.put 方法在插入新元素之前，将首先检查队列是否处于中间状态（步骤 A）。如果是，那么有另一个线程正在插入元素（在步骤 C 和 D 之间）。此时当前线程不会等待其他线程执行完成，而是帮助它完成操作，并将尾节点向前推进一个节点（步骤 B）。然后，它将重复执行这种检查，以免另一个线程已经开始插入新元素，并继续推进尾节点，直到它发现队列处于稳定状态之后，才会开始执行自己的插入操作。

由于步骤 C 中的 CAS 将把新节点链接到队列尾部，因此如果两个线程同时插入元素，那么这个 CAS 将失败。在这样的情况下，并不会造成破坏：不会发生任何变化，并且当前的线程只需重新读取尾节点并再次重试。如果步骤 C 成功了，那么插入操作将生效，第二个 CAS（步骤 D）被认为是一个“清理操作”，因为它既可以由执行插入操作的线程来执行，也可以由其他任何线程来执行。如果步骤 D 失败，那么执行插入操作的线程将返回，而不是重新执行 CAS，因为不再需要重试——另一个线程已经在步骤 B 中完成了这个工作。这种方式能够工作，因为在任何线程尝试将一个新节点插入到队列之前，都会首先通过检查 tail.next 是否非空来判断是否需要清理队列。如果是，它首先会推进尾节点（可能需要执行多次），直到队列处于稳定状态。

15.4.3　原子的域更新器

程序清单 15-7 说明了在 ConcurrentLinkedQueue 中使用的算法，但在实际的实现中略有区别。在 ConcurrentLinkedQueue 中没有使用原子引用来表示每个 Node，而是使用普通的 volatile 类型引用，并通过基于反射的 AtomicReferenceFieldUpdater 来进行更新，如程序清

单 15-8 所示。

程序清单 15-8 在 ConcurrentLinkedQueue 中使用原子的域更新器

```
private class Node<E> {
    private final E item;
    private volatile Node<E> next;

    public Node(E item) {
        this.item = item;
    }
}

private static AtomicReferenceFieldUpdater<Node, Node> nextUpdater
        = AtomicReferenceFieldUpdater.newUpdater(
                Node.class, Node.class, "next");
```

原子的域更新器类表示现有 volatile 域的一种基于反射的“视图”，从而能够在已有的 volatile 域上使用 CAS。在更新器类中没有构造函数，要创建一个更新器对象，可以调用 newUpdater 工厂方法，并制定类和域的名字。域更新器类没有与某个特定的实例关联在一起，因而可以更新目标类的任意实例中的域。更新器类提供的原子性保证比普通原子类更弱一些，因为无法保证底层的域不被直接修改——compareAndSet 以及其他算术方法只能确保其他使用原子域更新器方法的线程的原子性。

在 ConcurrentLinkedQueue 中，使用 nextUpdater 的 compareAndSet 方法来更新 Node 的 next 域。这个方法有点繁琐，但完全是为了提升性能。对于一些频繁分配并且生命周期短暂的对象，例如队列的链接节点，如果能去掉每个 Node 的 AtomicReference 创建过程，那么将极大地降低插入操作的开销。然而，几乎在所有情况下，普通原子变量的性能都很不错，只有在很少的情况下才需要使用原子的域更新器。（如果在执行原子更新的同时还需要维持现有类的串行化形式，那么原子的域更新器将非常有用。）

15.4.4 ABA 问题

ABA 问题是一种异常现象：如果在算法中的节点可以被循环使用，那么在使用“比较并交换”指令时就可能出现这种问题（主要在没有垃圾回收机制的环境中）。在 CAS 操作中将判断“V 的值是否仍然为 A？”，并且如果是的话就继续执行更新操作。在大多数情况下，包括本章给出的示例，这种判断是完全足够的。然而，有时候还需要知道“自从上次看到 V 的值为 A 以来，这个值是否发生了变化？”在某些算法中，如果 V 的值首先由 A 变成 B，再由 B 变成 A，那么仍然被认为是发生了变化，并需要重新执行算法中的某些步骤。

如果在算法中采用自己的方式来管理节点对象的内存，那么可能出现 ABA 问题。在这种情况下，即使链表的头节点仍然指向之前观察到的节点，那么也不足以说明链表的内容没有发生改变。如果通过垃圾回收器来管理链表节点仍然无法避免 ABA 问题，那么还有一个相对简单的解决方案：不是更新某个引用的值，而是更新两个值，包括一个引用和一个版本号。即使这个值由 A 变为 B，然后又变为 A，版本号也将是不同的。AtomicStampedReference(以及

AtomicMarkableReference) 支持在两个变量上执行原子的条件更新。AtomicStampedReference 将更新一个“对象－引用”二元组，通过在引用上加上“版本号”，从而避免[⊖] ABA 问题。类似地，AtomicMarkableReference 将更新一个“对象引用 - 布尔值”二元组，在某些算法中将通过这种二元组使节点保存在链表中同时又将其标记为“已删除的节点”。[⊜]

小结

非阻塞算法通过底层的并发原语（例如比较并交换而不是锁）来维持线程的安全性。这些底层的原语通过原子变量类向外公开，这些类也用做一种“更好的 volatile 变量”，从而为整数和对象引用提供原子的更新操作。

非阻塞算法在设计和实现时非常困难，但通常能够提供更高的可伸缩性，并能更好地防止活跃性故障的发生。在 JVM 从一个版本升级到下一个版本的过程中，并发性能的主要提升都来自于（在 JVM 内部以及平台类库中）对非阻塞算法的使用。

⊖ 在实际中，无论如何，理论上计数器都应该这样包装。

⊜ 许多处理器都提供了各种二元的 CAS（CAS2 或 CASX）操作，用于对一些“指针 - 整数”二元组进行操作，从而使这种操作的效率得到极大提高。从 Java 6 开始，在 AtomicStampedReference 并没有使用这种 CAS(即使平台支持这种操作)。(这种二元 CAS 与 DCAS 不同，DCAS 能在内存的两个互不相关的位置上执行操作，而在编写本书时，还没有处理器实现了 DCAS。)